Sovereign: SRS and Master Build Plan

A 3D interactive leader-election simulator for Distributed Computing

---

1. Project Overview

Sovereign is a real distributed system plus a cinematic 3D dashboard. Five or more independent Python processes ("nodes") exchange real network messages and elect a leader using the **Bully** and **Ring** algorithms. When the sovereign (leader) falls, the realm holds an election, and a 3D kingdom shows every message, crash and coronation live.

One-line pitch: *Crash the king, watch the kingdom elect a new one, and see exactly how many messages it cost.*

Why it is strong for a mini project

- The distributed part is real (separate processes, real sockets, real failures), so it is not just an animation.
- The visuals are 3D and interactive, so it stands out in a demo.
- It produces measurable results (message counts, election time), so it supports a proper analysis section.

2. Goals and Scope

Goals

- Implement Bully and Ring leader election correctly over real network communication.
- Provide a 3D web dashboard to observe and control the system.
- Measure and compare both algorithms (messages, election time, rounds).
- Make it easy to run (one command) and easy to explain in a viva.

In scope: leader election, heartbeat failure detection, fault injection, live visualization, analytics, replay of past elections.

Out of scope: user login, cloud deployment, persistent consensus (Paxos/Raft), multi-machine support (listed as a stretch goal).

3. Users and Use Cases

- **Presenter (you):** runs demos, kills nodes, switches algorithms, shows analytics.
- **Examiner:** watches the election, asks why a message was sent, checks the comparison.

Key use cases: start the cluster, kill or revive a node, assassinate the leader, switch algorithm, run Chaos mode, run a benchmark, replay an election, export results.

4. Functional Requirements

Cluster and nodes

- FR-1: The system starts N nodes (default 5, configurable 3 to 9 and adjustable live from a dashboard slider) as separate OS processes, each with its own port and numeric ID.
- FR-2: Nodes communicate over TCP using newline-delimited JSON messages.
- FR-3: Each node keeps its own state: ID, status (ALIVE, DEAD, ELECTING), current leader, algorithm in use, logical clock.
- FR-4: Nodes detect leader failure through heartbeats (ping every 2 s, timeout 3 s, both configurable).

Algorithms

- FR-5: Bully: ELECTION to higher IDs, OK replies, COORDINATOR broadcast, with a timeout when no OK arrives.
- FR-6: Ring: ELECTION token travels around the logical ring collecting alive IDs, then a COORDINATOR token announces the winner.
- FR-7: Algorithm can be switched at runtime from the UI (applied to the next election).
- FR-8: A revived node with a higher ID triggers a new election in Bully mode (the classic "bully takes over" behavior).
- FR-9: Handle simultaneous detection by several nodes without electing two leaders.

Fault injection

- FR-10: Kill or revive any node.
- FR-11: Assassinate the current leader.
- FR-12: Chaos mode: random crashes and revivals at a set interval.
- FR-13: Network delay and message-drop sliders per cluster.
- FR-14: Kill the leader mid-election to demonstrate fault tolerance.

Visualization (3D)

- FR-15: Render each node as a 3D castle on an island or terrain, with roads or glowing links between nodes.
- FR-16: Render each message as an animated particle traveling along the link, colored by type.
- FR-17: Leader castle wears a glowing crown; dead castles collapse with smoke and embers.
- FR-18: Camera supports orbit, zoom, cinematic auto-orbit, and "follow the election" mode.
- FR-19: Day and night theme with matching lighting.

Analytics and logging

- FR-20: Live message log with filters by type and node.
- FR-21: Per-election stats: messages sent, duration, rounds, initiator, winner.
- FR-22: Bully vs Ring comparison charts across many elections.
- FR-23: Benchmark mode: run 20 elections per algorithm automatically and plot the results.
- FR-24: Replay timeline: scrub back through a past election.
- FR-25: Export results as JSON or CSV.

Dynamic node scaling (node-count slider)

- FR-26: A slider in the dashboard sets the node count from 3 to 9 while the system is running; the gateway spawns or stops node processes to match.
- FR-27: A joining node gets the next free ID and port, announces itself to its peers, and learns the current leader. In Bully mode, a joining node with the highest ID triggers a new election, as a revived node does.
- FR-28: Lowering the count removes the highest-ID node first. The node leaves gracefully and notifies its peers; if it was the leader, an election starts.
- FR-29: Peer lists and the Ring order update on every membership change through a MEMBERSHIP message, and an election in progress restarts safely with the new membership.
- FR-30: The 3D scene animates the change: new castles rise and the circle re-spaces, removed castles sink away, and each election is stored with the node count it ran at.

5. Non-Functional Requirements

- **Performance:** 60 FPS on a normal laptop with 9 nodes; UI event latency under 100 ms.
- **Reliability:** a crashing node must never crash the gateway or other nodes.
- **Usability:** a first-time viewer understands the screen in under one minute (tooltips and a guided tour).
- **Portability:** runs on Windows, macOS and Linux with Python 3.11+ and Node 20+.
- **Maintainability:** typed code, clear folder layout, tests for both algorithms.
- **Accessibility:** keyboard shortcuts, reduced-motion option, readable contrast.

6. System Architecture

```
[ Browser: React + Three.js ]
        ^   |
  WebSocket |  REST (commands)
        |   v
[ Gateway / Orchestrator: FastAPI ]
   - spawns and supervises node processes
   - collects events, stores them in SQLite
   - forwards commands (kill, revive, switch algorithm)
        ^   |
  events    | control
        |   v
[ Node 1 ] <-TCP-> [ Node 2 ] <-TCP-> [ Node 3 ] <-TCP-> [ Node 4 ] <-TCP-> [
Node 5 ]
   (real election messages travel directly between nodes)
```

Key design rule: election messages go node to node directly. The gateway only observes (each node sends it a copy of every event) and sends control commands. This keeps the distributed part honest and easy to defend in a viva.

7. Technology Stack

Frontend

- **React 18 + Vite + TypeScript:** fast, typed, industry standard.
- **Three.js via React Three Fiber (R3F) + drei:** declarative 3D scenes in React, with orbit controls, text, and helpers.

- **@react-three/postprocessing:** bloom glow, vignette, depth of field for a cinematic look.
- **GSAP:** camera fly-throughs and timed sequences. **Framer Motion:** UI panel transitions.
- **Zustand:** small, fast state store for live events.
- **Tailwind CSS + shadcn/ui:** consistent, professional UI components.
- **Recharts:** analytics charts.
- **Howler.js:** optional sound effects (crown chime, crash, messenger whoosh).

Backend

- **Python 3.11 + asyncio:** every node is an async TCP server and client. Raw sockets keep the distributed concepts visible.
- **FastAPI + Uvicorn:** the gateway, with WebSocket streaming and REST commands.
- **SQLite (via SQLModel):** stores elections and events for replay and analytics.
- **Pydantic:** validated message schemas.
- **pytest + pytest-asyncio:** algorithm tests.

Tooling: `ruff` and `mypy` for Python, ESLint and Prettier for the frontend, and one launcher script (`python run.py`) that starts everything. Docker Compose is optional.

Why this stack: Python is the easiest language for socket-level distributed code, and React Three Fiber is the most productive way to build a polished 3D interface that is still beginner-friendly.

8. Message Protocol

All messages are one JSON object per line over TCP:

```
{
  "type": "ELECTION",
  "from": 3,
  "to": 5,
  "election_id": "e-0042",
  "algorithm": "bully",
  "payload": {},
  "lamport": 17,
  "ts": 1767000000.123
}
```

Message types

- `HEARTBEAT` and `HEARTBEAT_ACK`: failure detection.
- `ELECTION`, `OK`, `COORDINATOR`: Bully.
- `RING_ELECTION` (payload holds the list of alive IDs) and `RING_COORDINATOR`: Ring.
- `CONTROL`: kill, revive, set algorithm, set network delay, and MEMBERSHIP updates that carry the new peer list and ring order (gateway to node).
- `EVENT`: node to gateway, a copy of each sent or received message plus state changes.

Every message carries a **Lamport timestamp**, which makes event ordering in the replay correct and gives you an extra distributed-computing concept to show.

9. Algorithm Specifications

Bully

1. A node that detects leader failure starts an election and sends ELECTION to all higher IDs.
2. If any higher node answers OK, the starter waits for a COORDINATOR message (with a timeout; if none arrives, it restarts).
3. If no OK arrives within the timeout, the node declares itself leader and sends COORDINATOR to all.
4. A node receiving ELECTION from a lower ID replies OK and starts its own election.
5. Worst case is O(n^2) messages (lowest node detects first); best case is O(n).

Ring

1. The initiator creates an ELECTION token with its own ID and sends it to its next alive neighbor, skipping dead nodes after a timeout.
2. Each node appends its ID and forwards the token.
3. When the token returns to the initiator, it picks the highest ID as leader and sends a COORDINATOR token once around the ring.
4. Cost is roughly 2n messages per election, so O(n).

Duplicate elections: each election has an ID. If two nodes start at once, nodes keep the election with the higher initiator ID and drop the other, so only one leader results.

Counting messages: the gateway counts every ELECTION, OK, COORDINATOR and ring token per election ID. Heartbeats are counted separately so they do not distort the comparison.

10. 3D Scene and Animation Design

World: a floating island kingdom at dusk or dawn, with low-poly mountains, water with gentle waves, instanced trees, drifting clouds, fireflies at night, and a skybox with a moving sun or moon.

Nodes as castles: a procedural low-poly castle (built from simple meshes, so no heavy model files are required) with the node ID on a floating banner. States:

- ALIVE: warm lit windows, small flag.
- LEADER: golden glow, floating rotating crown, light beam into the sky, particle sparkles.
- ELECTING: pulsing blue ring around the castle.
- DEAD: tilt and collapse animation, smoke, embers, grey desaturation.

Messages: glowing orbs traveling along curved paths between castles. Colors: ELECTION orange, OK green, COORDINATOR gold, HEARTBEAT faint white (toggleable). A trail effect shows direction. Ring mode shows a single token orb circling the ring.

Cinematic moments

- Leader dies: slow-motion, camera pushes in, shockwave, screen shake.
- New leader crowned: camera flies to the winning castle, crown descends, bloom flash, brief fanfare.
- Chaos mode: lightning strikes random castles.

Camera modes: free orbit, auto-orbit showcase, follow-election, top-down diagram view (useful for explaining to examiners).

Performance: instanced meshes, baked or simple lighting, particle pooling, and a "Low graphics" toggle.

11. UI/UX Design

- **Layout:** full-screen 3D canvas with floating glass panels.
  - Top: banner showing the current sovereign, algorithm switch, and theme toggle.
  - Left: node list with status, quick kill and revive.
  - Right: message scroll (live log) and election history.
  - Bottom: control dock (Assassinate, Chaos, Benchmark, node-count slider, speed, delay and drop sliders).

    - Drawer: Analytics (charts), Replay timeline, Settings.
- **Visual style:** glassmorphism panels, gold and deep-navy palette, serif display font for titles (e.g. Cinzel) and a clean sans-serif (Inter) for data, subtle motion on every state change.
- **Interaction:** click a castle for a detail card (state, last heartbeat, messages sent), hover to highlight links, keyboard shortcuts (K kill leader, C chaos, Space pause).
- **Onboarding:** a 30-second guided tour on first load.
- **Empty and error states:** clear messages if the gateway is offline, with a reconnect indicator.

12. Folder Structure

```
sovereign/
  run.py                  # starts gateway + nodes
  README.md
  backend/
    gateway/              # FastAPI app, websocket, REST, SQLite
    node/                 # node.py, bully.py, ring.py, heartbeat.py,
protocol.py
    tests/
  frontend/
    src/
      scene/              # Island, Castle, Crown, Messenger, Camera, Effects
      ui/                 # panels, dock, charts, replay
      store/              # zustand stores
      lib/                # websocket client, types
  docs/                   # SRS, report, diagrams, viva notes
```

13. Execution Plan

Phase 1: Core distributed system (3 to 4 days)

- Node process with TCP server and client, JSON protocol, Lamport clock.
- Heartbeat and failure detection.
- Bully algorithm with tests. Then Ring with tests.
- Terminal logging to verify behavior.

Phase 2: Gateway (2 days)

- FastAPI app that spawns nodes, receives events, streams over WebSocket, exposes kill, revive, set-algorithm commands.
- SQLite storage and message counting per election, plus a scale endpoint that adds or removes nodes at runtime.

Phase 3: Frontend foundation (2 days)

- Vite + React + TS setup, WebSocket client, Zustand store.
- Basic 2D debug view to prove the data flow end to end.

Phase 4: 3D scene (4 to 5 days)

- Island, castles, roads, lighting, sky.
- State animations (leader, electing, dead), message orbs, camera modes.
- Post-processing and cinematic moments.

Phase 5: UI and analytics (3 days)

- Panels, control dock, log, history.
- Charts, benchmark mode, replay timeline, export.

Phase 6: Polish and testing (2 to 3 days)

- Chaos mode, sounds, tour, low-graphics mode.
- Edge cases (simultaneous elections, kill during election, rapid revive).
- Performance pass and bug fixing.

Phase 7: Documentation and viva prep (2 days)

- Report with architecture, algorithms, results, screenshots.
- Demo script and rehearsal.

Total: about 3 weeks of focused work. If time is short, drop sound, replay and Chaos mode first, and keep the core 3D scene plus both algorithms.

14. Testing Plan

- **Unit tests:** Bully and Ring state machines with a mocked network.
- **Integration tests:** start 5 real nodes, kill the leader, assert a single new leader within a time limit.
- **Edge cases:** two simultaneous detectors, leader dies mid-election, highest node revives, all but one node dead, message drops.

- **Benchmark:** 20 elections per algorithm at N = 3, 5, 7, 9, with results charted to confirm the O(n^2) vs O(n) behavior.
- **Frontend:** manual checklist for each node state, plus FPS check.

15. Demo Script (about 5 minutes)

1. Run `python run.py` and open the page. Show the kingdom and explain castles equal nodes.
2. Click a castle to show its state. Explain heartbeats.
3. Press **Assassinate**. Walk through the Bully election as orbs fly.
4. Show the log and Lamport timestamps.
5. Switch to Ring, repeat, and point at the message counts.
6. Kill the leader mid-election to show fault tolerance.
7. Open Analytics and run the benchmark. Show the comparison chart.
8. Finish with Chaos mode as the closing visual.

16. Likely Viva Questions

- Why is Bully O(n^2) in the worst case and Ring O(n)?
- What happens if two nodes detect the failure at the same time?
- How do you detect failure, and what are the limits of timeouts (slow node vs dead node)?
- What if the network drops the COORDINATOR message?
- Why did you use TCP and not UDP?
- What is the purpose of the Lamport clock in your design?
- Which algorithm would you use in production, and why? (Mention Raft and ZooKeeper-style election as real-world examples.)
- How would this change across multiple machines?

17. Master Build Prompt

Paste the following into your AI coding assistant (Claude Code is a good fit), one phase at a time.

```
You are a senior full-stack engineer. Build "Sovereign", a 3D interactive
leader-election simulator for a Distributed Computing mini project.

GOAL
Real Python node processes elect a leader using the Bully and Ring algorithms
over real TCP sockets. A React + Three.js web app shows the nodes as 3D
castles,
messages as animated orbs, and provides controls and analytics.

BACKEND
- Python 3.11, asyncio. Each node is a separate process: async TCP server and
  client, newline-delimited JSON messages, Pydantic schemas, Lamport clock.
- Node state: id, port, status (ALIVE/DEAD/ELECTING), leader_id, algorithm.
- Heartbeat failure detection (ping 2s, timeout 3s, configurable).
- Implement Bully (ELECTION, OK, COORDINATOR) and Ring (token collects alive
IDs,
  then COORDINATOR token). Support runtime algorithm switch, election IDs to
  resolve simultaneous elections, and a revived higher-ID node taking over in
  Bully mode.
- Fault injection via CONTROL messages: kill, revive, set delay, set drop
rate. Runtime scaling: the gateway adds or removes nodes on request (a scale
endpoint that spawns or stops node processes), and every node updates its
peer list and ring order through MEMBERSHIP messages when membership changes.
- Every node sends an EVENT copy of each message and state change to the
gateway.
- Gateway: FastAPI + Uvicorn. Spawns and supervises nodes, streams events
over
  a WebSocket, exposes REST endpoints (kill, revive, assassinate leader, set
  algorithm, chaos on/off, benchmark), stores elections and events in SQLite
  (SQLModel), counts messages per election (heartbeats counted separately).
- run.py starts the gateway and nodes with one command.
- pytest + pytest-asyncio tests for both algorithms, including edge cases.

FRONTEND
- React 18 + Vite + TypeScript, Tailwind + shadcn/ui, Zustand, Framer Motion,
  GSAP, Recharts.
- React Three Fiber + drei + @react-three/postprocessing (bloom, vignette).
- Scene: floating island kingdom, procedural low-poly castles (no heavy
assets),
  water, instanced trees, clouds, day/night with sun/moon and fireflies.
- Castle states: alive (lit windows), leader (golden glow, floating crown,
light
  beam), electing (pulsing ring), dead (collapse, smoke, embers).
```

```
- Messages as glowing orbs with trails along curved paths. Colors: ELECTION
  orange, OK green, COORDINATOR gold, HEARTBEAT faint white (toggleable).
- Cinematic moments: slow-mo and shockwave when the leader dies, camera fly-
to
  and crown descent on coronation. Camera modes: orbit, auto-orbit, follow
  election, top-down.
- UI: glass panels. Top banner (current sovereign, algorithm switch, theme),
  left node list, right live log and election history, bottom control dock
  (Assassinate, Chaos, Benchmark, node-count slider 3 to 9 with animated
castles joining and leaving, speed, delay, drop sliders), analytics drawer
  with Bully vs Ring charts, replay timeline, CSV/JSON export.
- Fonts: Cinzel for titles, Inter for data. Palette: deep navy and gold.
- Keyboard shortcuts, reduced-motion and low-graphics options, onboarding
tour,
  reconnect indicator.

QUALITY BAR
- 60 FPS with 9 nodes, event latency under 100 ms.
- Strict typing, linting (ruff, mypy, ESLint), clear README.
- Work phase by phase: (1) nodes + algorithms + tests, (2) gateway,
  (3) frontend data flow, (4) 3D scene, (5) UI + analytics, (6) polish.
  After each phase, show how to run and verify it before continuing.

Start with Phase 1 only.
```

Tip: after each phase, run it, test it, and only then paste the next phase instruction. Building in order keeps bugs small and makes your viva explanation easy.